

A

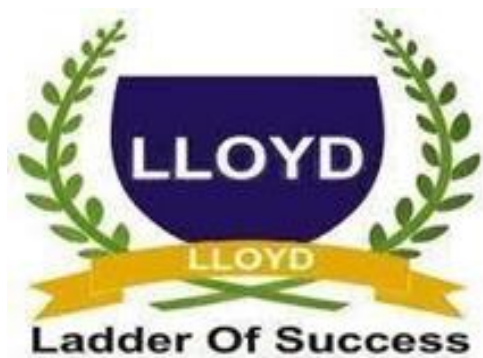
SYNOPSIS REPORT

On

RETRIEVAL-AUGMENTED GENERATION (RAG) APPLICATION

Submitted in partial fulfilment of the requirements of the third semester of

MASTER OF COMPUTER APPLICATIONS



Submitted By

Divya Chauhan (2501530140034)

Submitted To

Dr. Gaurav Jindal Sir

NAME OF THE INSTITUTION

SESSION 2026-2027

INDEX

S. No.	CONTENT	PAGE NO.
1	Abstract	1
2	Objectives	2
3	Literature Survey	3
4	Tools / Hardware / Software Required	4
5	Design & Methodology	5
6	Methodology	6
7	Implementation	6
8	Context DFD	7
9	Expected Results	8
10	Other References	8

1. ABSTRACT

Retrieval-Augmented Generation (RAG) is an approach for building intelligent question-answering applications by combining information retrieval with generative artificial intelligence. The proposed RAG Application is designed to allow a user to provide a collection of documents and then ask questions in natural language. Instead of depending only on information stored inside a language model, the application first searches the available document collection for relevant information and then uses the retrieved content as supporting context for generating an answer. This makes the system more suitable for document-based question answering, where the answer should be grounded in the information supplied by the user.

The proposed application focuses on creating a simple and practical workflow. Documents are uploaded through a user interface and processed into smaller text chunks. These chunks are converted into numerical representations called embeddings. The embeddings are stored in a vector database so that semantically similar information can be retrieved when a user submits a query. During question answering, the query is also represented in a form suitable for similarity search. The retriever identifies the most relevant document chunks and passes them to the generation component. The language model then produces a natural-language response.

The system is intended to reduce the difficulty of searching large documents manually. A user does not need to read every page to locate a particular fact. Instead, the user can ask a direct question and receive an answer based on the indexed material. The application can therefore be useful for study notes, project documentation, manuals, reports, policies, and other structured or semi-structured documents. The design also provides a foundation for future improvements such as conversation history, source citations, multiple document collections, user authentication, document management, and improved retrieval techniques.

The project demonstrates the integration of modern software development with artificial intelligence concepts. The main components include document ingestion, text processing, embedding generation, vector storage, semantic retrieval, prompt construction, and response generation. A web-based interface provides a convenient way to upload documents and interact with the question-answering system. The proposed solution emphasizes modularity so that individual components can be replaced or improved.

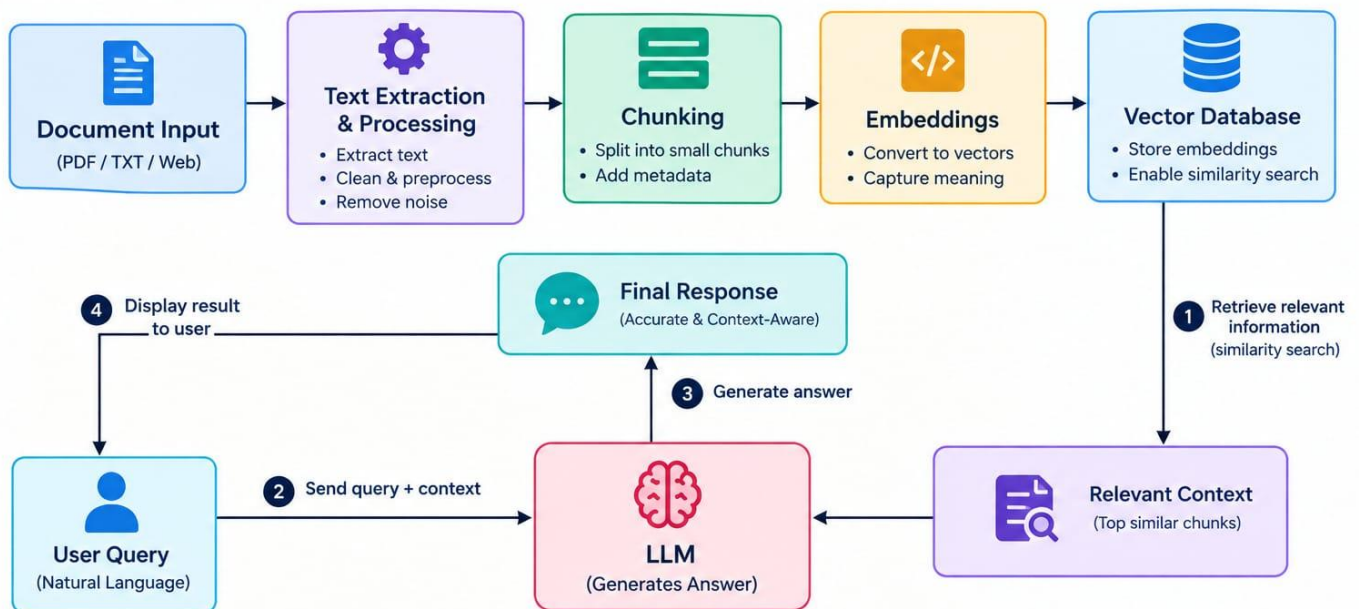


Figure 1: Abstract of RAG Application

2. OBJECTIVES

The primary objective of the proposed Retrieval-Augmented Generation (RAG) Application is to develop a document-based question-answering system that can retrieve relevant information from user-provided documents and use that information to generate meaningful responses. The project is designed with the following objectives:

1. To develop a web-based interface through which users can upload and manage documents for question answering.
2. To process uploaded documents by extracting text and dividing the content into manageable chunks.
3. To generate embeddings for document chunks so that textual information can be searched according to semantic similarity rather than only exact keywords.
4. To store document embeddings and associated text in a suitable vector database for efficient retrieval.
5. To accept natural-language questions from users and convert them into searchable representations.
6. To retrieve the most relevant document chunks for each user query.
7. To provide retrieved context to a language model so that generated answers are grounded in the available document content.
8. To design a simple interaction flow that can be used by students, researchers, employees, and other users who need to work with document collections.
9. To reduce the time and effort required for manually searching long documents.
10. To create a modular architecture in which document processing, retrieval, generation, backend services, and frontend components can be developed and maintained separately.
11. To provide a foundation for future features such as source references, chat history, multiple collections, authentication, document deletion, and improved retrieval methods.
12. To gain practical knowledge of Retrieval-Augmented Generation, embeddings, vector search, APIs, backend development, and AI application integration.

The project also aims to demonstrate that a useful AI application can be created by combining established software components in a clear pipeline. Rather than attempting to train a new large language model, the proposed system concentrates on retrieving relevant external information and using it as context during generation. This makes the project suitable for an academic implementation while still representing a modern AI application architecture.

3. LITERATURE SURVEY

Existing System

Traditional document search systems generally depend on keyword matching. In such systems, a user enters one or more words and the application searches for documents containing those words. Although keyword search can be effective for exact terms, it may fail when the user expresses the same concept using different words. It also requires the user to identify suitable search terms and manually examine the returned documents.

Conventional question-answering applications may also rely entirely on information already available to a language model. Such systems can generate fluent responses, but when the required information is contained in a private or newly provided document, the model may not have direct access to that information. A response can therefore be incomplete or not sufficiently grounded in the user's document collection.

Another common approach is to read documents manually and prepare answers. This can work for a small number of pages, but the process becomes time-consuming as document size increases. It is especially inconvenient when users repeatedly need to find different pieces of information from the same document collection.

Proposed System

The proposed RAG Application combines semantic retrieval with generative AI. Uploaded documents are first processed and divided into chunks. Embeddings are generated for these chunks and stored in a vector database. When a user submits a question, the application searches the vector database for chunks that are semantically related to the query. The retrieved chunks are then supplied to a language model as context for answer generation.

The proposed approach improves the interaction between users and documents because the user can ask questions in natural language rather than searching only for exact keywords. The retrieval stage also provides an explicit information source for the generation stage. The application can be designed so that retrieved content or source information can be displayed with the answer, helping the user understand where the response comes from.

Comparison

3.3 Comparison: Traditional LLM vs RAG System

Aspect	Traditional LLM	RAG System
 Data Source	Pre-trained/internal knowledge	External or user-provided documents
 Retrieval	No external retrieval	Semantic/vector retrieval
 Updates	Requires model update/retraining for new knowledge	Documents can be updated independently
 Answer Basis	Model knowledge	Retrieved context + LLM
 Use Case	General questions	Document-based question answering

Figure 3.1: Comparison of Traditional LLM and RAG System

4. TOOLS / HARDWARE / SOFTWARE REQUIRED

Hardware Requirements

The proposed application can be developed and tested on a standard personal computer or laptop. A high-end dedicated system is not mandatory for the basic implementation because document processing, web application logic, and vector retrieval can be handled by software services and APIs where appropriate.

- Processor: Intel Core i3/i5 or equivalent processor.
- RAM: Minimum 8 GB recommended for development and testing.
- Storage: At least 10 GB of free space for source code, dependencies, sample documents, and local development files.
- Internet: Stable internet connection for package installation and any external AI/API services used by the implementation.
- Display: Standard monitor suitable for web development and testing.

Software Requirements

- Operating System: Windows, Linux, or macOS.
- Programming Language: Python for the core RAG and backend logic.
- Frontend: HTML, CSS, and JavaScript or a suitable frontend framework if used in implementation.
- Backend/API: Python-based web framework such as FastAPI or Flask, depending on the final implementation.
- Document Processing: Libraries for reading and extracting text from supported document formats.
- Embedding Model/API: An embedding model or embedding API for converting text into vector representations.
- Vector Database: A vector-capable database for storing embeddings and performing similarity search.
- Language Model: A suitable generative language model/API for response generation.
- Development Tools: Code editor/IDE, terminal, package manager, Git, and GitHub.
- Browser: Modern browser such as Chrome, Edge, or Firefox for frontend testing.

Functional Requirements

- The system should allow document upload.
- The system should process and index document content.
- The system should accept natural-language questions.
- The system should retrieve relevant document content.
- The system should generate an answer using retrieved context.
- The system should display the response in an understandable format.

5. DESIGN

System Architecture

The proposed RAG Application follows a modular architecture consisting of a user interface, document ingestion layer, text processing layer, embedding component, vector database, retrieval component, prompt construction layer, and language model. The architecture separates the storage and retrieval of information from the generation of the final response.

When a document is uploaded, the ingestion component extracts its text. The extracted text is divided into smaller chunks so that relevant portions can be retrieved efficiently. Each chunk is converted into an embedding and stored along with suitable metadata. The query workflow begins when the user submits a question. The application converts the question into an embedding and searches the vector database for the most similar chunks. These chunks form the context supplied to the language model.

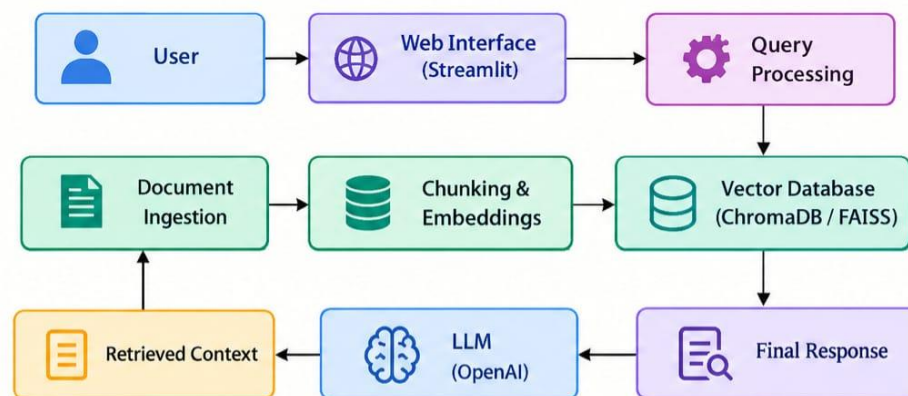


Figure 5.1: System Architecture of RAG Application

Data Flow

The main data flow begins with document input and ends with an AI-generated answer. During indexing, documents move from the upload interface to the processing component, then to chunking and embedding generation, and finally to vector storage. During query execution, the user's question is processed and used to retrieve relevant stored chunks. The retrieved information is combined with the question to create a prompt, which is passed to the language model. The generated response is then returned to the user interface.

Database Design

The vector database stores the embedding representation of each document chunk together with the original text or a reference to it. Metadata can include document name, chunk number, page information, upload information, and other identifiers. This structure allows the retrieval component to return both the semantic match and enough information to identify the source content.

06. METHODOLOGY

Document Ingestion

The first stage is document ingestion. The application accepts supported documents through the user interface. The backend receives the file, validates it, extracts readable text, and prepares the content for indexing. Different document formats may require different extraction libraries or processing methods.

Text Chunking

Large documents are not normally processed as one continuous block. The extracted text is divided into smaller chunks. Chunk size and overlap can be configured according to the document type and retrieval requirements. Overlapping sections can help preserve context when an important sentence spans two adjacent chunks.

Embedding Generation

Each chunk is converted into an embedding, which is a numerical representation of its semantic meaning. Similar pieces of text can have similar vector representations even when their exact words are different. These vectors provide the basis for semantic retrieval.

Vector Storage

The generated embeddings are stored in a vector database. The stored record can contain the vector, text chunk, document identifier, and metadata. The vector database provides similarity-search functionality that allows the application to locate the most relevant chunks for a query.

7. IMPLEMENTATION

Frontend Layer

The frontend provides the user-facing interface of the application. It can contain a document upload section, document status area, question input field, submit control, and answer display area. The interface should clearly separate document management from question answering so that the user can understand the current state of the application.

Backend Layer

The backend manages document upload, text extraction, chunking, embedding generation, vector storage, retrieval, and communication with the language model. API endpoints can be created for uploading documents, checking indexed documents, submitting questions, and returning generated answers. Proper validation and error handling should be included so that invalid files or failed requests do not terminate the application.

Retrieval Component

The retrieval component is responsible for selecting useful information from the indexed document collection. Similarity search can be configured to return a fixed number of top results. The retrieval process is an important part of the application because the quality of the generated answer depends strongly on the relevance of the context supplied to the language model.

Generation Component

The generation component receives the user's question together with retrieved context. A suitable prompt format can instruct the language model to answer using the provided information. If source references are retained in metadata, the application can also display supporting document information with the generated response.

8. CONTEXT DFD

The Context Data Flow Diagram represents the RAG Application as a single major system interacting with external entities. The primary external entity is the user. The user provides documents and questions to the application and receives processed answers as output. Internally, the application performs document processing, vector storage, retrieval, and answer generation.

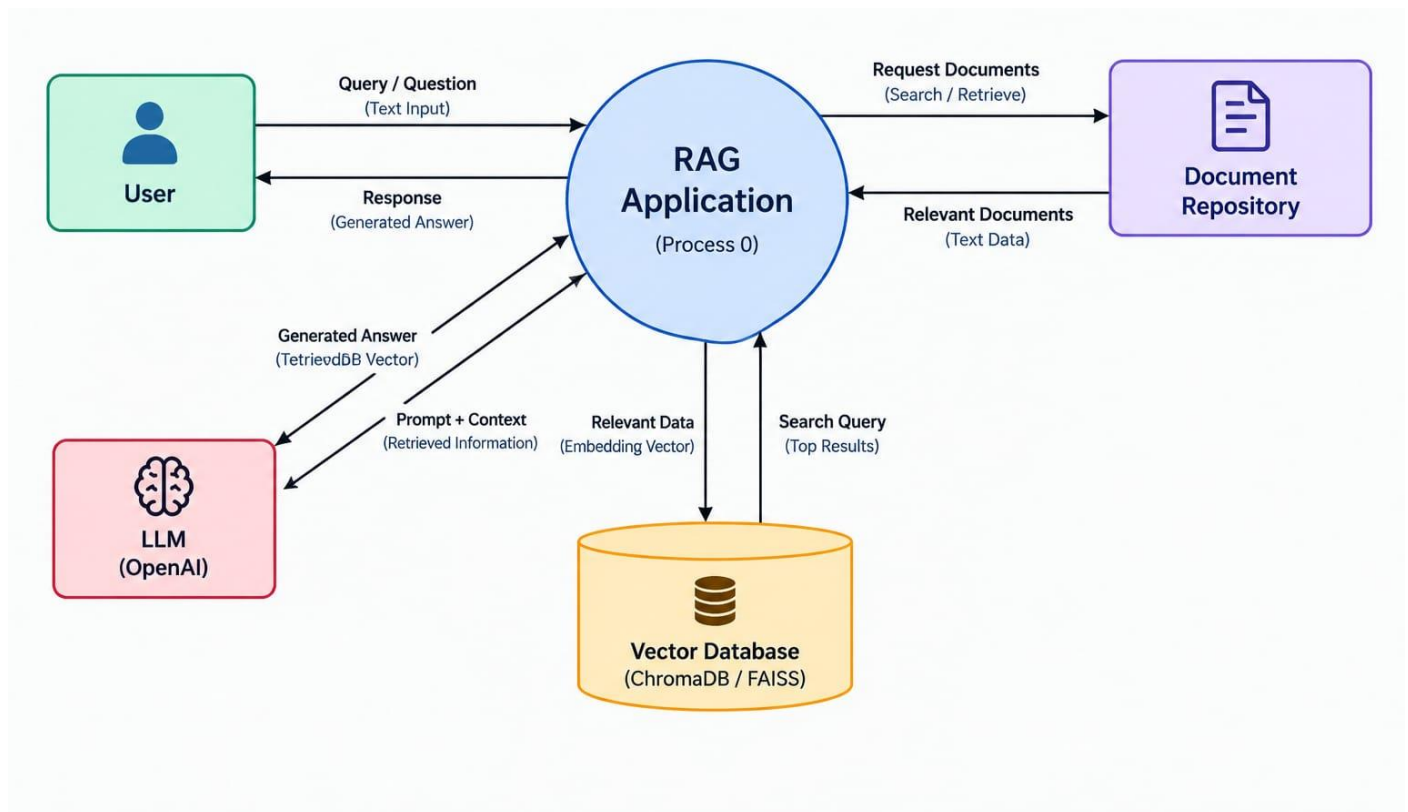


Figure 2.1: Context Data Flow Diagram of the Proposed RAG Application

The User interacts with the application by uploading documents and entering questions in natural language. The uploaded documents are processed by the system and the relevant information is stored for later retrieval. When a user submits a query, the application searches the available document information to identify the most relevant content.

The Document Repository acts as the source of document information. The RAG Application requests relevant documents or text data from the repository during the retrieval process. The retrieved information is then passed to the next stages of the RAG pipeline.

9. EXPECTED RESULTS

The expected result of the project is a functional RAG-based document question-answering application. The system should allow a user to upload supported documents, process their content, and ask questions related to the uploaded information. For relevant questions, the application is expected to retrieve appropriate document chunks and generate a clear response based on the retrieved context.

- Successful document upload and processing.
- Creation and storage of embeddings for document chunks.
- Semantic retrieval of relevant information from the document collection.
- Generation of natural-language answers using retrieved context.
- Simple and understandable web-based interaction.
- Reduced manual effort in locating information within long documents.
- Modular architecture suitable for future expansion.
- Practical demonstration of Retrieval-Augmented Generation concepts.

The final quality of the system will depend on document extraction, chunking strategy, embedding quality, retrieval configuration, and the selected language model. Therefore, testing and iterative tuning will be used to improve the relevance and usefulness of generated responses.

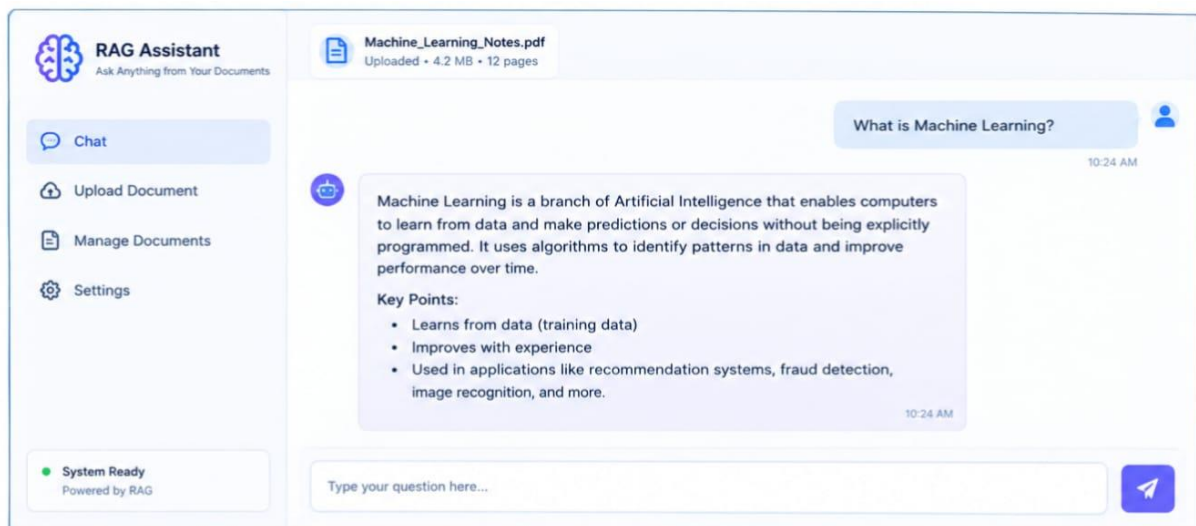


Figure 9.1: Expected Output of RAG Application

11. OTHER REFERENCES

1. Lewis, P. et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Proceedings of the 34th Conference on Neural Information Processing Systems (NeurIPS), 2020.
2. Vaswani, A. et al., “Attention Is All You Need,” Advances in Neural Information Processing Systems, 2017.
3. Relevant official documentation for the selected Python framework, embedding model/API, vector database, and language-model API used during implementation.
4. Python documentation and package documentation consulted during development and testing.
5. Git documentation and repository documentation used for source-code version control and project management.